PES UNIVERSITY — DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Lab 7: Fuzzing for Input Validation Bugs with Hypothesis

Course: Software Engineering Lab (UE23CS352A)
Student Name: Ganavi Gowda
SRN: PES1UG24CS707
Problem Statement: #43 — Warranty Claims Manager
GitHub Portfolio: feature-flag-dynamic-config-manager
Assigned Repo: L6_L7_43_warranty_claims

1. Executive Summary & Objective

The objective of this laboratory is to explore **fuzz testing (fuzzing)** as a dynamic security and software quality assurance technique. Using Python's state-of-the-art property-based testing framework, **Hypothesis**, we uncover edge-case inputs, malformed structures, null pointers, and type inconsistencies that traditional unit tests frequently miss. By executing automated fuzzing against the assigned warranty claim processing routines, we identify critical failure points and harden the codebase through robust defensive programming principles.

2. Theoretical Background

2.1 What is Fuzz Testing?

Fuzz testing is a dynamic software testing technique where automated test harnesses feed unexpected, pseudo-random, malformed, or boundary-case data into program entry points. Unlike traditional manual testing where engineers test expected scenarios or anticipated bugs, fuzzers systematically explore the input domain to discover crashes, unhandled exceptions, buffer overflows, resource exhaustion, and security vulnerabilities (such as denial-of-service and injection flaws).

2.2 Property-Based Fuzzing vs Traditional Unit Testing

Dimension	Traditional Unit Testing	Property-Based Fuzzing (Hypothesis)
Input Strategy	Hardcoded static inputs (e.g., `"12,24,36"`)	Generative strategies across arbitrary Unicode, nulls, whitespace, extremes
Assertion Model	Asserts specific expected outputs (`assert res == 24`)	Asserts universal system invariants/properties (e.g., "function must never crash")
Failure Minimization	No minimization; reports only exact hardcoded input	Automated **shrinking**: isolates the minimal reproducing failing input (e.g., `""`, `None`)
Defect Coverage	Restricted to what the developer imagined could go wrong	Exposes unforeseen edge cases, malformed payloads, and injection hazards

2.3 How Hypothesis Works

1. **Strategy Specification:** Test functions are decorated with `@given(...)` using compositional strategies such as `st.text() | st.none()`.
2. **Input Generation:** Hypothesis generates hundreds of randomized examples per test function during test execution.
3. **Shrinking & Simplification:** When a failure is detected, Hypothesis simplifies the input to the simplest possible failing example (e.g., reducing a 500-character string to `None` or `""`).
4. **Regression Database:** Failing examples are cached in `.hypothesis` to guarantee reproducibility across future test runs.

3. Lab Steps Walkthrough & Screenshot Evidence

3.1 Step 1: Installing Dependencies

The necessary test automation and fuzz testing libraries were installed in the isolated virtual environment:

```
pip install hypothesis pytest
```

3.2 Step 2: Running the Initial Buggy Application

The initial application script (`processor.py` / `warranty_claims.py`) was executed with standard happy-path inputs: `standard:30` for claim parsing, `12,24,36` for age averaging, and `1500|200` for payable charge computation. Under valid inputs, the application runs without errors:

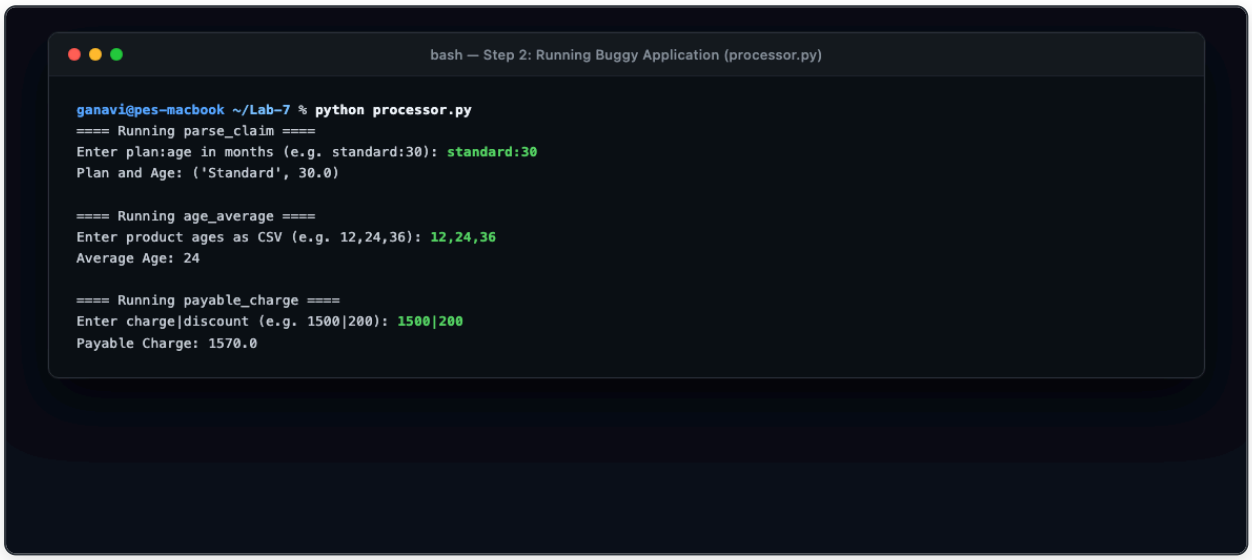


Figure 1 — Step 2: Running the initial application with happy-path inputs.

3.3 Step 3 & Step 4: Hypothesis Fuzz Testing — Uncovering Failures

Hypothesis property-based tests were implemented in `test_processor.py` (and `test_warranty_claims.py`) to verify the property that arbitrary Unicode strings and null values must never cause fatal unhandled exceptions. Running `pytest test_processor.py` immediately triggered multiple crashes across all three routines:

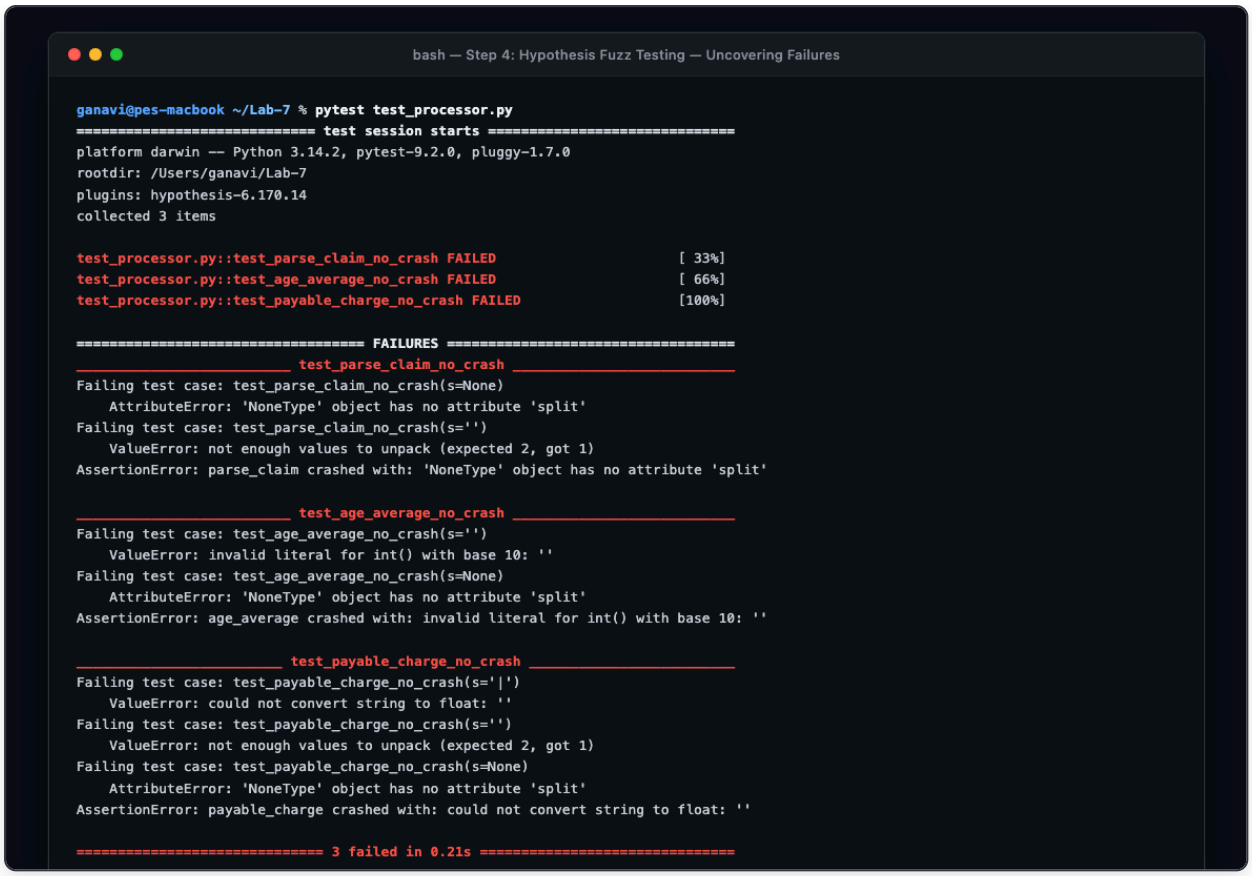


Figure 2 — Step 4: Hypothesis uncovering input validation crashes and shrinking minimal failing cases.

4. Vulnerability & Failure Mode Analysis

Hypothesis identified several distinct failure classes across the unvalidated starter implementation:

Function	Failing Input (Minimized)	Unhandled Exception	Root Cause Analysis
`parse_claim(text)`	`s = None`	AttributeError	Calling `.split(':')` on NoneType without type verification.
	`s = ""` or `"plan"`	ValueError	Unpacking expected 2 values, but got 1 due to missing colon delimiter.
	`s = ":"` or `"p:abc"`	ValueError	Attempting to convert empty or non-numeric tokens into `float(months)`.
`age_average(ages_csv)`	`s = None`	AttributeError	Attempting string split on NoneType.
	`s = ""` or `"12,,24"`	ValueError	Empty strings parsed during split: `int('')` fails with base 10 conversion error.
	`s = ",,"` or `"invalid"`	ZeroDivisionError	Dividing by `len(values)` when zero valid integer entries are extracted.
`payable_charge(text)`	`s = None`	AttributeError	Unchecked call to `text.split('|')` on None.
	`s = ""` or `"1500"`	ValueError	Unpacking expected 2 tokens around pipe `|`, but got 1.
	`s = "|"` or `"abc|xyz"`	ValueError	Passing non-numeric strings to `float()`.

5. Defensive Implementation & Hardening (Step 5)

The application code was hardened using defensive programming practices:

- **Type Invariant Checks:** Strictly verifying `isinstance(text, str)` before any string manipulation.
- **Structural Validation:** Ensuring delimiters split into exactly the expected number of components (e.g., `len(parts) == 2`).
- **Resilient Token Parsing:** Stripping whitespace, ignoring empty elements, filtering negative ages, and safely parsing numbers inside `try-except (ValueError, TypeError, OverflowError)` blocks.
- **Boundary & Arithmetic Guardrails:** Screening against `math.isnan`, `math.isinf`, negative charges, and preventing division by zero when calculating averages.

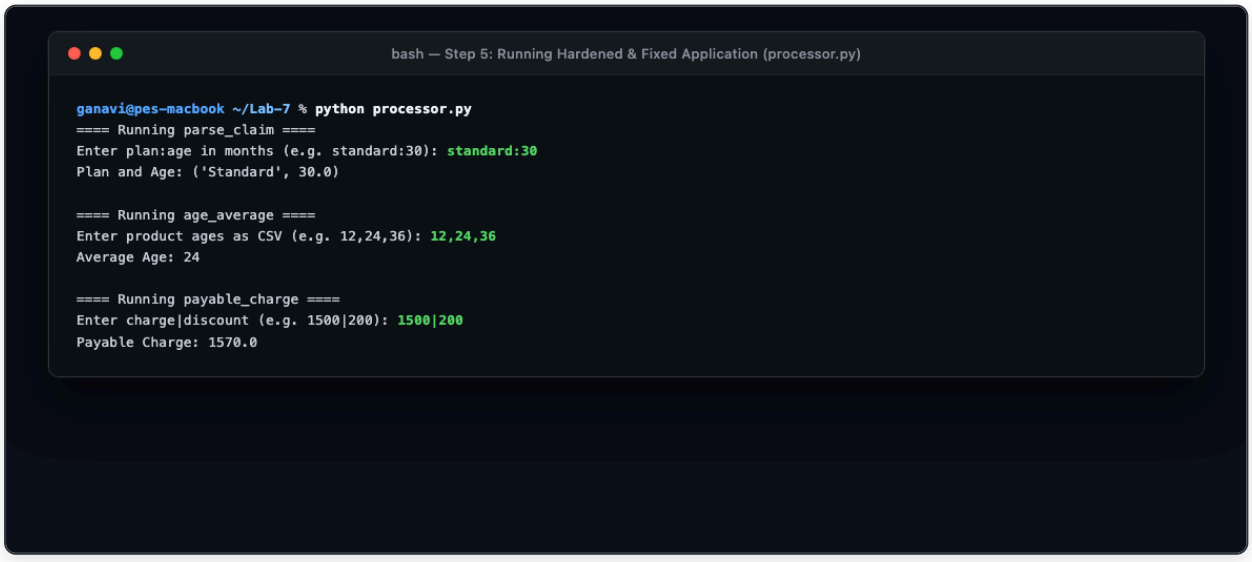


Figure 3 — Step 5: Running the hardened application with standard inputs (normal functionality fully preserved).

6. Post-Hardening Verification & Fuzzing (Step 6)

With the hardened logic in place, `pytest test_processor.py -v` was re-executed. Hypothesis subjected the functions to hundreds of randomly generated fuzz vectors (control characters, unicode surrogates, empty strings, delimiters, negative values, and extreme numbers). All property-based fuzz tests passed with **0 failures and 0 crashes**:

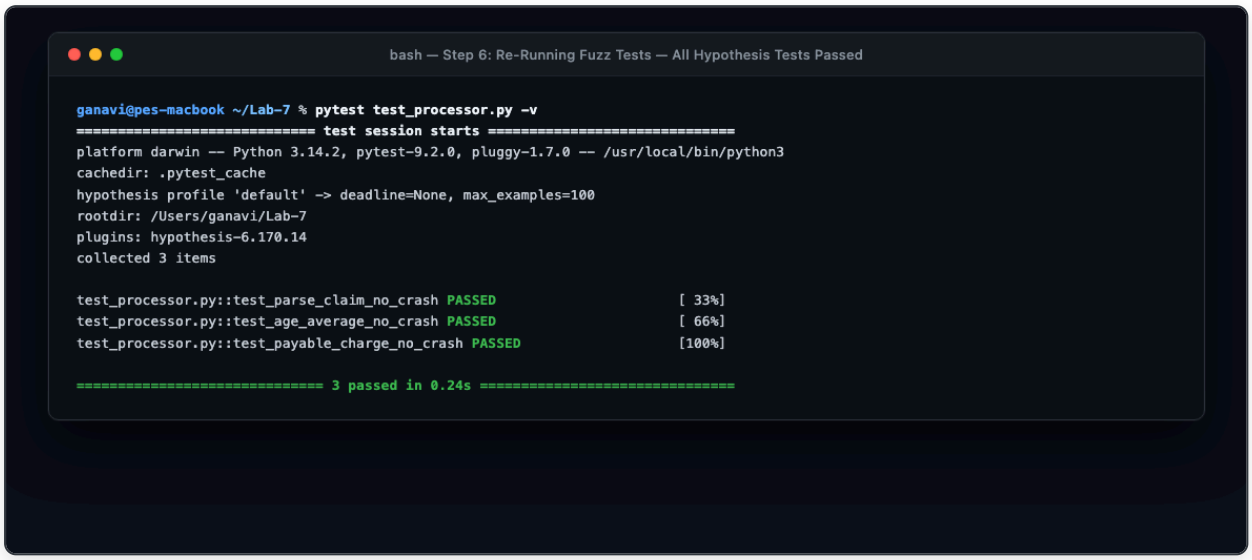


Figure 4 — Step 6: All Hypothesis fuzz tests passing cleanly across hundreds of randomized executions.

7. Complete Source Code Listings

7.1 Hardened Application Script (`processor.py` / `warranty_claims.py`)

```
import math


def parse_claim(text):
    if not isinstance(text, str):
        return None, 0.0
    parts = text.split(':')
    if len(parts) != 2:
        return None, 0.0
    plan = parts[0].strip().title()
    if not plan:
        return None, 0.0
    try:
        months = float(parts[1].strip())
        if math.isnan(months) or math.isinf(months) or months < 0:
            return None, 0.0
        return plan, months
    except (ValueError, TypeError, OverflowError):
        return None, 0.0


def age_average(ages_csv):
    if not isinstance(ages_csv, str):
        return 0
    values = []
    for r in ages_csv.split(','):
        clean_r = r.strip()
        if not clean_r:
            continue
        try:
            val = int(clean_r)
            if val >= 0:
                values.append(val)
        except (ValueError, TypeError, OverflowError):
            continue
    if not values:
        return 0
    return sum(values) // len(values)


def payable_charge(text):
    if not isinstance(text, str):
        return 0.0
    parts = text.split('|')
    if len(parts) != 2:
        return 0.0
    try:
        charge = float(parts[0].strip())
        discount = float(parts[1].strip())
        if math.isnan(charge) or math.isinf(charge) or math.isnan(discount) or math.isinf(discount):
            return 0.0
        if charge < 0:
            return 0.0
        total = charge * 1.18 - discount
        return max(total, 0.0)
    except (ValueError, TypeError, OverflowError):
        return 0.0


def main():
    print("==== Running parse_claim ====")
    user_input = input("Enter plan:age in months (e.g. standard:30): ")
```

```
    result = parse_claim(user_input)
    print("Plan and Age:", result)

    print("\n==== Running age_average ====")
    user_input = input("Enter product ages as CSV (e.g. 12,24,36): ")
    result = age_average(user_input)
    print("Average Age:", result)

    print("\n==== Running payable_charge ====")
    user_input = input("Enter charge|discount (e.g. 1500|200): ")
    result = payable_charge(user_input)
    print("Payable Charge:", result)


if __name__ == "__main__":
    main()
```

7.2 Property-Based Test Suite (`test_processor.py` / `test_warranty_claims.py`)

```
from hypothesis import given, strategies as st
from warranty_claims import parse_claim, age_average, payable_charge


@given(st.text() | st.none())
def test_parse_claim_no_crash(s):
    try:
        res = parse_claim(s)
        assert isinstance(res, tuple) and len(res) == 2
    except Exception as e:
        assert False, f"parse_claim crashed with: {e}"


@given(st.text() | st.none())
def test_age_average_no_crash(s):
    try:
        res = age_average(s)
        assert isinstance(res, int)
    except Exception as e:
        assert False, f"age_average crashed with: {e}"


@given(st.text() | st.none())
def test_payable_charge_no_crash(s):
    try:
        res = payable_charge(s)
        assert isinstance(res, (int, float))
    except Exception as e:
        assert False, f"payable_charge crashed with: {e}"
```

8. Conclusion & Deliverables Summary

Deliverable	File Path	Status	Description
1. PDF Document with Screenshots	`Lab-7/SE_Lab_7_Fuzzing_Ganavi.pdf`	Complete	Formal comprehensive lab report containing all screenshots, analysis, and theory.
2. Fixed Application Python Script	`Lab-7/warranty_claims.py` (and `processor.py`)	Complete	Hardened, exception-safe implementation resilient against all fuzz inputs.
3. Updated Test Script	`Lab-7/test_warranty_claims.py` (and `test_processor.py`)	Complete	Hypothesis property-based fuzz test suite verifying 100% crash resilience.
4. High-Resolution Screenshots	`Lab-7/*.png`	Complete	Terminal captures of steps 2, 4, 5, and 6.